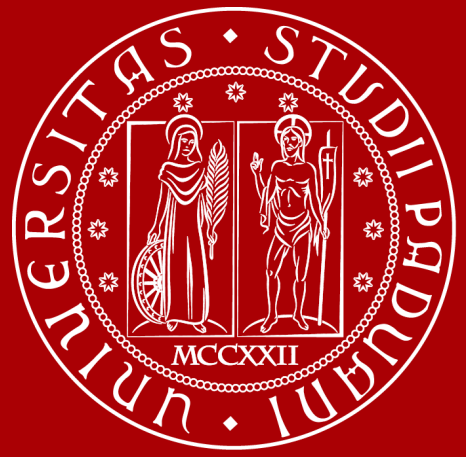


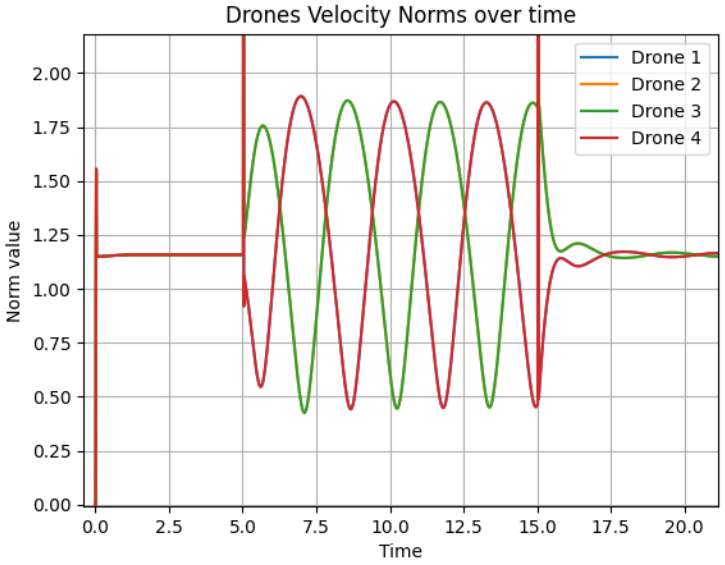
UNIVERSITÀ
DEGLI STUDI
DI PADOVA

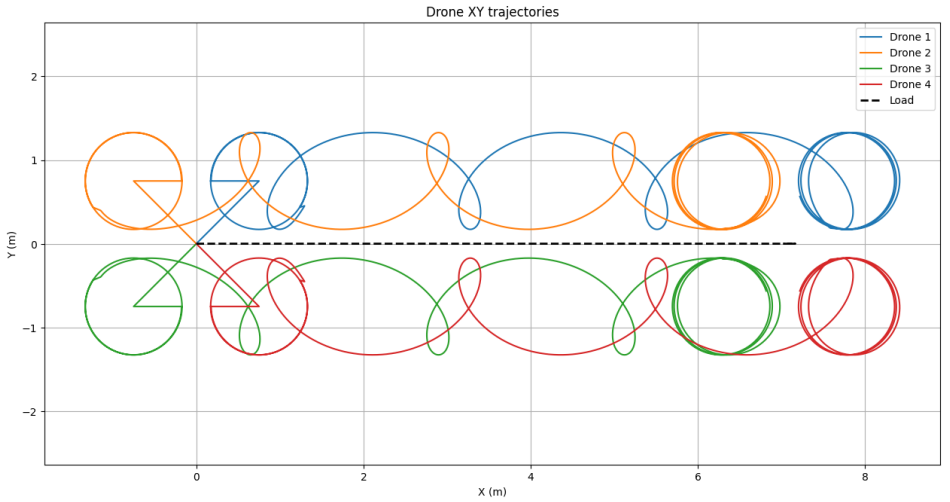
RL Cooperative Transportation using fixed-wings drones

Meeting 3

Updates

- I was able to replicate most of Sofia's thesis results
- Still facing some problems with the optimizer, some anomalies in the results and I have some questions
- Decisions regarding the RL architecture





Motivation for RL

- Decentralization: each drone takes its own decisions. No need for communication or synching

- Computational Complexity
  - Optimizer: ~3-4 ms/step (on my laptop)*
  - Trained NN inference: A structure similar to the one used in [1] would take <<1ms ** on any modern GPU(like a Jetson Nano)

*The FMU alone takes ~0.01-0.02 ms/step

**The theoretical compute time, based on FLOP count and peak GPU throughput, is on the order of microseconds. However, in practice, the total inference latency would be dominated by GPU overhead

Candidate RL architectures/Formulations
Formulation 1:

- Action Space: $\lambda \in R^n$
  - Keep wrench controller and nullspace matrix. Run decentralized

$$f(t) = f_g(t) + \underline{f_\lambda(t)},$$

$f_\lambda(t) = N(H,t) \cdot \underline{\lambda(t)}$

- Reward function:
  - Penalize $\|\dot{p}R_i\| < \varepsilon$ (would be interesting to see if agent comes up with the same strategy)
  - Can also warm start it with Imitation learning: reward for mimicking the optimizer output
- Load tracking guaranteed by construction ($G \cdot N = 0$)
- Doesn't handle desynchronization
- A possible extension would be adding more terms to the reward function to account for fixed-wings dynamics(e.g. minimum turning radius)

Candidate RL architectures/Formulations
Formulation 2: Residual RL

- Action Space: $\delta f \in R^{3n}$
  - Keep wrench controller and optimizer. Run decentralized

$$f(t) = f_g(t) + f_\lambda(t) + \underline{\delta f(t)}$$

- Reward function:
  - Penalize load tracking error + Penalize $\|\delta f\|$
- Handles desynchronization
- Stage 1: Train formulation 2
- Stage 2: Remove optimizer and train formulation 1
- Could also work the other way around(would be better for simulation time) or combined in one phase

Candidate RL architectures/Formulations
Another approach

- Action Space: Full force vector (and its derivative).

$$f(t) = f_g(t) + f_\lambda(t),$$

- Reward function:
  - Imitation learning : reward for mimicking the wrench controller + optimizer output
  - Can add Residual RL on top to fine tune behavior(based on IL shortcomings)
  - Probably doing RL from scratch would be too difficult

Start ideal, then progressively add noise, desynchronization and domain randomization

Candidate RL architectures/Formulations

- Wrap FMU inside Gym environment
- Centralized Training Decentralized Execution (CTDE)*
- Shared/Centralized critic with access to global state with a Parameter-Shared decentralized Actor
- Multi-Agent Proximal Policy Optimization (MAPPO)

- These choices(at least as a starting point) are due to their widespread adoption across cooperative multi-agent literature

*Can be a fully decentralized actor-critic if we only care about the agent velocity norm(formulation 1)

Doubts regarding the system model, wrench controller and the optimizer

- Are the desired forces injected directly into the cables?
- Do we calculate RL by just integrating RL_dot
- Static controller initialization of Fg
- How does load go back to x,y origin in the 2a(no feedback)?
- Attachment points and their effect
- Which Hamiltonian cycle
- Wpos and Wvel are harmful?
- Low level controller
- Optimizer just makes the drone fly in a straight line

References

[1] J. Zeng, A. Matoses Gimenez, E. Vinitsky, J. Alonso-Mora, and S. Sun, “Decentralized aerial manipulation of a cable-suspended load using multi-agent reinforcement learning,” in *Proceedings of the Conference on Robot Learning (CoRL)*, Seoul, South Korea, 2025. [Online]. Available: https://proceedings.mlr.press/v305/zeng25a.html

UNIVERSITÀ
DEGLI STUDI
DI PADOVA